Social Mining – Practical Session 2025

Cédric BOSCHER
cedric.boscher@univ-st-etienne.fr

December 12, 2025

Introduction

This practical session will focus on graph mining with Python. We will use the NetworkX library. The code will be written in Python 3.7 or higher with Networkx 2.6.0 or higher.

Work Submission

At the end of the practical session, you are expected to submit your code (either .py or .ipynb files) on moodle, in the Data Mining Course, under the **Social Mining - Practical Work Submission** form. The submission will grant you 1 point on your evaluation grade.

Technical Environment

This Practical session will focus graph mining with Python. We will use the NetworkX and Scikit-Learn. The code will be written with Python 3.

As mentioned in the mail you received before the practical session, you have two options :

1. Use your personal computer , provided you can run Python with a virtual environment on it without issues. You can try creating a virtual environment by following the quick-start section of the documentation and testing the installation of a library (for instance: pip install gensim).
2. Alternatively, you may use Google Colab to create a Python notebook in a remotely hosted virtual environment. Please note that this option requires a Google account (apologies for any inconvenience).

Option 1 - Virtual Environment on your personal machine machine

1. First, make sure that you have a correct version of Python installed.
   In the console, type `python --version`.
2. Then, you can create a virtual environment with `venv`, for instance:
   `python -m venv <your_environment_name>`
3. Then, activate your environment as follows:
   `source <your_environment_name>/bin/activate`

4. Then install the libraries:

   ```
   pip install networkx scikit-learn matplotlib node2vec
   ```

Option 2 - Use Google Colab

1. **Open Google Colab:**
   - Visit https://colab.research.google.com and log in with your Google account.
2. **Start a New Notebook:**
   - Click on the *"File"* menu and select *"New notebook"*.
   - Alternatively, you can click *"New Notebook"* directly on the homepage.
3. **Rename Your Notebook:**
   - Click on the default title (usually "Untitled") at the top left and type your desired name.
4. **Add and Edit Cells:**
   - Use the *"+ Code"* or *"+ Text"* buttons to add code and text cells as needed.
   - Code cells run Python code, while text cells can contain formatted text using Markdown.
5. **Save and Access:**
   - The notebook automatically saves to your Google Drive, so you can access it anytime in the *"Colab Notebooks"* folder.

1 Creating a Graph

1. Start python and import `networkx as nx`

2. To create an empty graph, simply type `G = nx.Graph()`

3. You can add nodes and edges with: `G.add`$_node(1)or$

3. 3. `List of nodes: list(G.nodes)`

4. `List of edges: list(G.edges)`

5. `Degree of a node: G.degree give the list of degrees by node and G.degree[1] gives the degree of the node 1 in G`

Write a function that generates a random Erdos-Renyi graph. It will take as input the number of nodes n of the graph and probability p that an edge exist between any pair of nodes

You can obtain the largest component of the graph with `max(nx.connected_components(G), key=len)`. In the particular case where $n \cdot p = 1$, what can you say about the size of the largest connected component? You can plot the size of the largest component as a function of the number of nodes in the graph.

Compare the execution time and the behaviour of your function with the erdos renyi generator of networkx `nx.erdos_renyi_(n, p)`

2 Loading a Graph

You can read and write a graph in a lot of different ways:

- Download the graph0.txt file on Moodle.
- Read the graph with `nx.read edgelist("./graph0.txt")`

2.1 Updating a Graph

- Add a the node "10" and an edge between "5" and "10"

2.2 Plotting a Graph

Basically, you can plot graphs with the following:

- install matplotlib: `pip install matplotlib`
- import the library with: import matplotlib.pyplot as plt
- plot the lists x vs y: `plt.figure() // plt.plot(x, y) // plt.show()`

For more information, see the matplotlib documentation.

The easiest way to draw a graph is to use `nx.draw(G, with_labels = True)` then show it with `plt.show()`. As you already know, many layouts are available for drawing graphs. You can find them at Networkx drawing.

1. Try to plot the graph you loaded and modified below.

3 Degree Distributions

1. Plot the graph and the degree distribution of :
   - The graph you loaded and modified
   - The karate_club_graph, using `nx.karate_club_graph()`
   - A LFR (Lancichinetti–Fortunato–Radicchi) graph, generated as follows :
     `G = nx.LFR_benchmark_graph(250, 3, 1.5, 0.1, average_degree=5, min_community=20, seed=10)`
2. These graphs are benchmark generated graphs that have a-priori known communities, and that are used as benchmarks for the evaluation of community detection algorithms. In the next section, you will explore such a graph for detecting communities.

   Which specificity can you see when you plot the Degree distribution of a LFR Graph, compared to the graphs you plotted below ?

4 Community Detection

A typical task in graph mining is to detect communities which are groups are nodes that are densely connected together and not densely connected with the rest of the graph. During the lecture, we discussed some methods. Networkx offers many different algorithms to uncover the community structure of a graph.

In this section, you will learn how to explore the ground-truth communities of a LFR Graph, how to run community detection algorithms, and how to measure the quality of detected communities with Normalized Mutual Information.

1. First, use the LFR Graph that you generated as follows in the previous section :

```
G = nx.LFR_benchmark_graph(250, 3, 1.5, 0.1, average_degree=5, min_community=20, seed=10)
```

2. Acceed the ground-truth communities of the graph with `nx.get_node_attributes(lfr,'community')`.

   The returned ground-truth is a dictionnary, which keys correspond to nodes, and values correspond to a set of nodes forming a community. The communities are disjoint, meaning that each node is contained in one single community.

3. With the function below, assign a community label to each node, i.e. a numerical ID that labels the community the node belongs to. The latter function assigns an arbitrary label to the community, that corresponds to the ID of its last node. By doing so, verify that the groundtruth contains 3 distinct communities.

```
def groundtruth_communities_to_labels(communities):
    """
    Inputs :
        - communities : the detected communities
        - G : the Graph

    Output : An array containing all nodes and their community labels.

    The index corresponds to the ID of the node,
    and the value corresponds to the node's community label

    For instance : [ 1, 3, 1] means that :

    the nodes 0 and 2 belong to a community named "1",
    while the node 1 corresponds to a community named "3".
    """

    for key, value in communities.items():
        groundtruth[key] = list(value)[-1]
    return list(groundtruth.values())
```

4. Now, you will apply a community detection algorithm to your graph, in order to compare the detected communities with the ground-truth.

   By refering to the NetworkX documentation, apply a function from `nx.community` that finds the best partitions for the Louvain Community Detection Algorithm.

5. Then, assign a community label to each node. Note that the data structure of the output of the Louvain Algorithm is slightly different from the ground-truth, which means that the labelling function has to be adapted as follows:

```
def detected_communities_to_labels(communities, G):
    """
    Inputs :
        - communities : the detected communities
        - G : the Graph

    Output : An array containing all nodes and their community labels.

    The index corresponds to the ID of the node,
    and the value corresponds to the node's community label

    For instance : [ 1, 3, 1] means that :

    the nodes 0 and 2 belong to a community named "1",
    while the node 1 corresponds to a community named "3".
    """

    labels = [-1]*len(G)

    for i in range(len(communities)):
        for j in list(communities[i]):
            labels[j] = i

    return labels
```

How many communities were detected by the Louvain Community Detection Algorithm in your case ?

6. Compare the found partition with the ground-truth partition. You may use the Mutual Information of two partitions for example (see sklearn NMI Normalized Mutual Information score).

   The obtained results should highlight that the Louvain algorithm is not optimal in your case.

7. During the lecture, you discussed how to divise a graph into communities with Divisive clustering on Edge-Betweenness. You may use the `nx.community.girvan_newman` algorithm, that is based on this principle. By referring to the documentation, implement the Girvan Newman algorithm in such a way that it generates the expected number of communities.

   Compute the NMI for the generated communities, and verify that it performs better than Louvain.

5 Optional - Graph Embedding

1. Install `node2vec` with pip. With the help of the documentation, fit a Node2vec model on your graph and compute the embeddings of the nodes of the LFR Graph.

2. Apply the clustering method of your choice in order to detect communities based on node embeddings, and compute the NMI between your clustering and the ground-truth.

6 Optional - Display Graphs

Write a function that displays graphs with the following requirements :

- The size of the nodes must be their betweenness centrality
- The colour must be the community they belong to.